

st125333_Lab_Assignment2.1

September 10, 2025

[27]: *#Sat Naing Tun 125333*

1 Neural Network

A neural network is inspired by the structure of the human brain. Its basic building block is a neuron, which takes inputs, performs computations, and produces an output. Each neuron has an associated activation function that determines whether it should “fire” based on its inputs.

Training a neural network requires to understand a few naming conventions:

1. Architecture and Layers A simple neural network consists of an input layer, a hidden layer and an output layer. For deep neural networks, the hidden layers should be more than one and possibly sometimes the number of input features are also larger.

Simple Neural Network x_1, x_2 and x_3 are the input features and y_1 and y_2 are the outputs. Here, we can say, the model architecture is receiving 3 features and 2 outputs(2 classes) in a classification model.

2. Parameters In between each layers, there are some linear functions which consist of trainable parameters called weights and biases. Basically, the training of a neural network is updating these weights and biases to achieve the objective function of the specific problem.

Linear function with parameters Let's assume that x is an input feature from a data, the linear function is such a way that the weight w multiple with the input feature x and sum up with the bias b . In below graph, we explicitly assign $w = 1/2$ and $b = 1$ for easy understanding, and it shows a linear line representing that function. This example is only happening in one neuron or one feature. However in neural networks, there will be a lot of similar operations happening in parallel. In such cases, the vectorization and matrix operations are used for faster and more efficient computations.

3. Activation Function The activation function introduces non-linearity to the network. It decides whether a neuron should be active or not based on its weighted inputs. Common activation functions include sigmoid, tanh, and Rectified Linear Unit (ReLU).

Deep Neural Network Architecture image referenced from: [Industry 4.0 Interoperability, Analytics, Security, and Case Studies](#)

4. Feedforward In the feedforward neural network, the calculation is only forward pass without updating any parameters yet. The idea is first we calculate the linear function using the input features, then again from the outputs of the linear function, an activation function is introduced in order to get the non-linear outputs. Then the activated outputs from the previous layer go to the next layer as the input of that layer. The process is generally repeated through all the layers until we get the final result y_i which is the predicted output. In feedforward, the predicted output may be some way different from the ground truth class.

5. Loss Function A loss function measures how far off the neural network's output is from the expected output(label or ground truth). It quantifies the network's performance and guides the learning process. Generally, in deep learning, minimizing the loss between the predicted output and the ground truth is the objective function of the model.

6. Optimizer An optimizer is an algorithm or a method used to adjust the parameters (weights and biases) of a neural network during training in order to minimize the loss function(objective function) or maximize the efficiency of the model. The optimizer helps in adjusting the learning rate, momentum and direction of the gradient calculation during training. Examples of optimizer:

1. Stochastic Gradient Descent(SGD)
2. Adam
3. RMSProp
4. Adagrad and so on...

7. Backpropagation Backpropagation is the process of updating the weights and biases in the network to minimize the loss function(objective function). It calculates the gradient of the loss with respect to the network's parameters and updates them using optimization algorithms like Gradient Descent.

Computational Graph

Derivation

Derivative Examples $f(x) = x^2 \gg \frac{\partial}{\partial x} f(x) = 2x$

$f(x) = x^3 \gg \frac{\partial}{\partial x} f(x) = 3x^2$

$f(x) = \log_e(x)$ or $f(x) = \ln(x) \gg \frac{\partial}{\partial x} f(x) = \frac{1}{x}$

$f(x) = e^x \gg \frac{\partial}{\partial x} f(x) = e^x$

$f(x) = 2x^2 + 3x + b \gg \frac{\partial}{\partial x} f(x) = 4x + 3$ (The derivative of a constant is always zero)

8. Training The network is trained using labeled data. During training, the input data is passed through the network, the output is compared to the expected output, and the loss is calculated. Backpropagation then adjusts the weights and biases to minimize the loss so that we will get the optimal weights and biases that fit with our problem.

Gradient Descent Gradient Descent is an optimization algorithm used in machine learning and deep learning to iteratively update the parameters of a model in order to minimize a loss function. The goal of the algorithm is to find the values of the model's parameters that result in the lowest possible value of the loss function, which in turn signifies the best fit of the model to the training data.

1. Initialization: The algorithm starts with an initial guess for the model's parameters.
2. Compute Loss: The current model's parameters are used to make predictions on the training data. The difference between these predictions and the actual target values (the loss) is calculated using a chosen loss function.
3. Compute Gradient: The gradient of the loss with respect to each parameter is computed. The gradient indicates the direction and magnitude of the steepest increase in the loss function. This step involves taking partial derivatives of the loss function with respect to each parameter.
4. Update Parameters: The parameters are updated by subtracting a fraction of the gradient from the current parameter values. This fraction is called the learning rate. The learning rate determines the step size in the parameter space and influences the speed and stability of convergence.
5. Repeat: Steps 2 to 4 are repeated for a certain number of iterations or until the change in the loss function becomes small (convergence criterion).

Gradient descent algorithm reference: Deep Learning Specialization by Andrew Ng from [Coursera](#).

```
<img src='img/gradientdescent.png' alt='Gradient Descent Algorithm' width="300" style="margin-right: 10px;" />
<img src='img/localoptima.png' alt='Local Optima' width="175" style="margin-right: 10px;" />
<img src='img/gradientdescent2.png' alt='Gradient Descent' width="400" />
```

9. Validation and Evaluation After training, the network's performance is evaluated using validation data to ensure it's not overfitting, underfitting, or the model is learning. Once satisfied, the network is tested on unseen data to measure its real-world performance. The testing process is sometimes called inferencing.

2 Dealing with Python Packages/ Libraries

Installing a library with PIP The command to install in your terminal is `pip3 install packagename`. You can find the related package name and it's version in <https://pypi.org/>.

Example with Numpy Library Before you start, please make sure that you have already installed Numpy library which is a handy package for scientific computing in Python.

To install Numpy, just simply use `pip3 install numpy` or `pip install numpy` in your terminal, or put an exclamation point in front `!pip3 install numpy` or `!pip install numpy` in your Jupyter cell.

To start using the library, simply import the library in the beginning of your Text editor or Jupyter cell.

```
import numpy as np
```

Notice that `np` is a shortcut name which you will use that throughout the implementation instead

of `numpy`. It is just a standard way of shortcut, however you can name it any shortcut name. It is recommended to use a standard form.

Some other useful libraries you might use in the future:

```
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image
from sklearn.model_selection import train_test_split    «    This is importing
train_test_split method from model_selection module from Scikit-Learn(sklearn) li-
brary.
```

Loading and Image Using PIL and Matplotlib Libraries

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image # Use for loading image into
from IPython.display import display
```

```
[2]: img = Image.open('img/python.png')
print("Image object: ", img)
img_data = np.array(img)
x, y, _ = img_data.shape
print("Original size: ", (x, y, _))
img = img.resize((int(x/4), int(y/4)))
print("Resized size", np.array(img).shape)
# img.show()
display(img)
```

```
Image object: <PIL.PngImagePlugin.PngImageFile image mode=RGBA size=1200x1315
at 0x1310990A420>
```

```
Original size: (1315, 1200, 4)
```

```
Resized size (300, 328, 4)
```

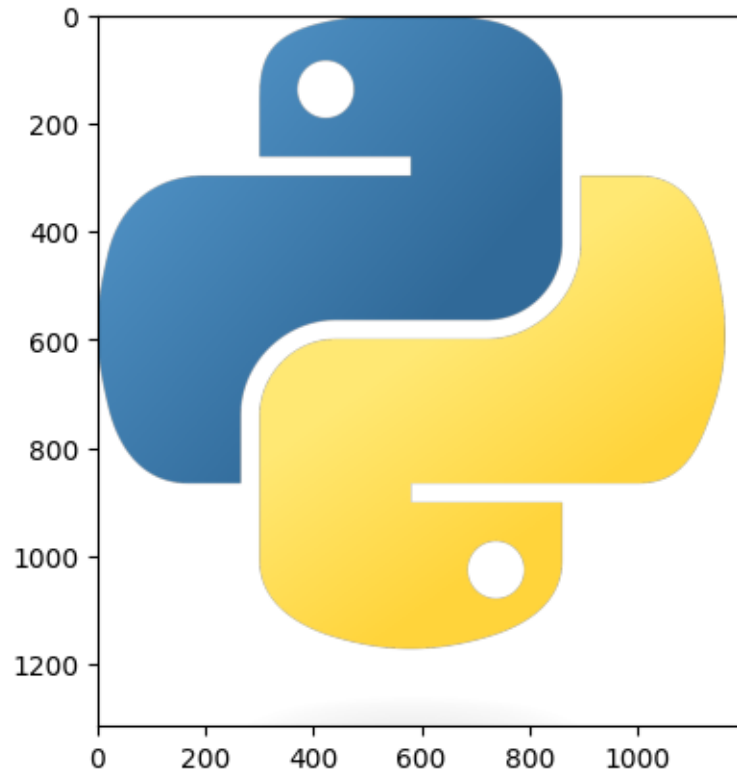


```
[3]: # Python program to read
      # image using matplotlib

      # importing matplotlib modules
      import matplotlib.image as mpimg
      import matplotlib.pyplot as plt

      # Read Images
      img = mpimg.imread('img/python.png')
      print(img.shape)
      height, width, _ = img.shape
      # Output Images
      # plt.figure(figsize=(height/72, width/72))
      plt.imshow(img)
      plt.show()
```

(1315, 1200, 4)



Loading an Image Using OpenCV To load the image, we will try a different method by using a library called OpenCV. First, we will need the installation of `opencv-python`.

`pip install opencv-python` or `conda install -c conda-forge opencv`

```
[4]: import cv2

# Load the image
image = cv2.imread("img/cat.png")

# # Display the image
cv2.imshow("Image", image)

# # Wait for the user to press a key
cv2.waitKey(0)

# # Close all windows
cv2.destroyAllWindows()
```

3 Vectorization and Matrix Operation

Different types of vector/matrix operations

1. The dot product (inner scalar product)
 $\mathbf{x1} \cdot \mathbf{x2}$
2. Elementwise product (Hadamard product)
 $\mathbf{x1} \circ \mathbf{x2}$
 $\mathbf{x1} \odot \mathbf{x2}$
3. The outer product
 $\mathbf{x1} \otimes \mathbf{x2}$

Comparing the running times

```
[5]: ### Importing libraries before writing your codes
import time
import numpy as np
import math

# x1 = [9, 2, 5, 0, 0, 7, 5, 0, 0, 0, 9, 2, 5, 0, 0]
# x2 = [9, 2, 2, 9, 0, 9, 2, 5, 0, 0, 9, 2, 5, 0, 0]

np.random.seed(27)
x1 = np.random.randint(500, 1000, 500)
x2 = np.random.randint(0, 500, 500)
print("x1[0]:", x1[0], " ", "x2[0]:", x2[0])

### CLASSIC DOT PRODUCT OF VECTORS IMPLEMENTATION ###
tic = time.time()
dot = 0
for i in range(len(x1)):
    dot += x1[i]*x2[i]
toc = time.time()
duration = toc - tic
# print ("dot = " + str(dot) + "\n ----- Computation time = " + str(1000*(toc -
    ↪tic)) + "ms")
# print ("dot ----- Computation time = " + str(1000*(toc - tic)) + "ms")
print(f"dot ----- Computation time = {duration * 1000:.6f} ms")

### CLASSIC OUTER PRODUCT IMPLEMENTATION ###
tic = time.time()
outer = np.zeros((len(x1),len(x2))) # we create a len(x1)*len(x2) matrix with
    ↪only zeros
for i in range(len(x1)):
    for j in range(len(x2)):
        outer[i,j] = x1[i]*x2[j]
toc = time.time()
```

```

duration = toc - tic
# print ("outer = " + str(outer) + "\n ----- Computation time = " +
    ↳str(1000*(toc - tic)) + "ms")
# print ("outer ----- Computation time = " + str(duration * 1000) + "ms")
print(f"outer ----- Computation time = {duration * 1000:.6f} ms")

### CLASSIC ELEMENTWISE IMPLEMENTATION ###
tic = time.time()
mul = np.zeros(len(x1))
for i in range(len(x1)):
    mul[i] = x1[i]*x2[i]
toc = time.time()
duration = toc - tic
# print ("elementwise multiplication = " + str(mul) + "\n ----- Computation
    ↳time = " + str(1000*(toc - tic)) + "ms")
# print ("elementwise multiplication ----- Computation time = " + str(1000*(toc
    ↳- tic)) + "ms")
print(f"elementwise multiplication ----- Computation time = {duration * 1000:.
    ↳6f} ms")

### CLASSIC GENERAL DOT PRODUCT IMPLEMENTATION ###
W = np.random.rand(3,len(x1)) # Random 3*len(x1) numpy array
tic = time.process_time()
gdot = np.zeros(W.shape[0])
for i in range(W.shape[0]):
    for j in range(len(x1)):
        gdot[i] += W[i,j]*x1[j]
toc = time.time()
# print ("gdot = " + str(gdot) + "\n ----- Computation time = " + str(1000*(toc
    ↳- tic)) + "ms")
print ("gdot ----- Computation time = " + str(1000*(toc - tic)) + "ms")

```

```

x1[0]: 519    x2[0]: 34
dot ----- Computation time = 0.000000 ms
outer ----- Computation time = 199.057102 ms
elementwise multiplication ----- Computation time = 0.000000 ms
gdot ----- Computation time = 1757440428915.3188ms

```

```

[6]: # x1 = [9, 2, 5, 0, 0, 7, 5, 0, 0, 0, 9, 2, 5, 0, 0]
     # x2 = [9, 2, 2, 9, 0, 9, 2, 5, 0, 0, 9, 2, 5, 0, 0]

```

```

np.random.seed(27)
x1 = np.random.randint(500, 1000, 500)
x2 = np.random.randint(0, 500, 500)
print("x1[0]:", x1[0], " ", "x2[0]:", x2[0])

### VECTORIZED DOT PRODUCT OF VECTORS ###

```



```

tic = time.time()
dot = np.dot(x1,x2)
toc = time.time()
# print ("dot = " + str(dot) + "\n ----- Computation time = " + str(1000*(toc -
↳ tic)) + "ms")
print ("dot ----- Computation time = " + str(1000*(toc - tic)) + "ms")

### VECTORIZED OUTER PRODUCT ###
tic = time.time()
outer = np.outer(x1,x2)
toc = time.time()
# print ("outer = " + str(outer) + "\n ----- Computation time = " +
↳ str(1000*(toc - tic)) + "ms")
print ("outer ----- Computation time = " + str(1000*(toc - tic)) + "ms")

### VECTORIZED ELEMENTWISE MULTIPLICATION ###
tic = time.time()
mul = np.multiply(x1,x2)
toc = time.time()
# print ("elementwise multiplication = " + str(mul) + "\n ----- Computation
↳ time = " + str(1000*(toc - tic)) + "ms")
print ("elementwise multiplication ----- Computation time = " + str(1000*(toc -
↳ tic)) + "ms")

### VECTORIZED GENERAL DOT PRODUCT ###
tic = time.time()
dot = np.dot(W,x1)
toc = time.time()
# print ("gdot = " + str(dot) + "\n ----- Computation time = " + str(1000*(toc
↳ - tic)) + "ms")
print ("gdot ----- Computation time = " + str(1000*(toc - tic)) + "ms")

```

```

x1[0]: 519    x2[0]: 34
dot ----- Computation time = 0.9989738464355469ms
outer ----- Computation time = 1.0004043579101562ms
elementwise multiplication ----- Computation time = 0.0ms
gdot ----- Computation time = 0.6976127624511719ms

```

Exercise 1

1. Create three matrices and print out the results:

$$\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 6 & 5 \end{pmatrix}, \mathbf{B} = \begin{pmatrix} 3 & 4 \\ 8 & 7 \end{pmatrix}, \mathbf{C} = \begin{pmatrix} 3 & 4 & 0 \\ 8 & 7 & 9 \end{pmatrix}$$

2. Calculate the following and print out the results:

1. $\mathbf{A} \cdot \mathbf{B}$ » Try 3 different styles using @ sign or `np.dot()` or `np.matmul()` functions. All three results should be the same.

2. $\mathbf{A} \cdot \mathbf{C}$
3. $2\mathbf{A}$
4. $2\mathbf{A}^T$ » To transpose the matrix, you can simply use `matrix.T` or `np.transpose()` function.
5. $3\mathbf{A}^T \cdot \mathbf{B}$ » Print out the result and shape.
6. $3\mathbf{A}^T \cdot \mathbf{C}$ » Print out the result and shape.
7. $\mathbf{A} \odot \mathbf{B}$
8. $\mathbf{A} \odot \mathbf{C}$ » This should show up error.

3. Comment the difference between $\mathbf{A} \cdot \mathbf{B}$ and $\mathbf{A} \odot \mathbf{B}$

```
[7]: ### Your code here
# 1. Create matrices
A=np.array([[2,1],[6,5]])
B=np.array([[3,4],[8,7]])
C=np.array([[3,4,0],[8,7,9]])
# print(A)
# print(B)
# print(C)
# A . B (Three different styles)
result1=A@B
print("A.B="+str(result1))
result2=np.dot(A,B)
print("A.B="+str(result2))
result3=np.matmul(A,B)
print("A.B="+str(result3))
# A . C
result=np.dot(A,C)
print("A.C="+str(result))
# 2A
result=2*A
print("2A="+str(result))
# 2A.T
result=2*A.T
print("2A.T="+str(result))
# 3A.T . B (Print out shape and result)
result=np.dot(3*A.T,B)
print("3A.T.B="+str(result))
print("Shape of 3A.T.B="+str(result.shape))
# 3A.T . C (Print out shape and result)
result=np.dot(3*A.T,C)
print("3A.T.C="+str(result))
print("Shape of 3A.T.C="+str(result.shape))
# A * B
result=A*B
print("A*B="+str(result))
```

```
# A * C (error)
result=A*C
print("A*B="+str(result))
```

```
A.B=[[14 15]
      [58 59]]
A.B=[[14 15]
      [58 59]]
A.B=[[14 15]
      [58 59]]
A.C=[[14 15 9]
      [58 59 45]]
2A=[[ 4 2]
     [12 10]]
2A.T=[[ 4 12]
       [ 2 10]]
3A.T.B=[[162 150]
         [129 117]]
Shape of 3A.T.B=(2, 2)
3A.T.C=[[162 150 162]
         [129 117 135]]
Shape of 3A.T.C=(2, 3)
A*B=[[ 6 4]
      [48 35]]
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[7], line 38
    35 print("A*B="+str(result))
    37 # A * C (error)
--> 38 result=A*C
    39 print("A*B="+str(result))

ValueError: operands could not be broadcast together with shapes (2,2) (2,3)
```

Expected Output:

```
A . B = [[14 15] [58 59]]
A . C = [[14 15 9] [58 59 45]]
2A = [[ 4 2] [12 10]]
2A.T = [[ 4 12] [ 2 10]]
3A.T . B = [[162 150] [129 117]]
Shape of 3A.T . B = (2, 2)
3A.T . C = [[162 150 162] [129 117 135]]
```

Shape of $3A.T \cdot C = (2, 3)$

$A * B = \begin{bmatrix} 6 & 4 \\ 48 & 35 \end{bmatrix}$

$A * C = \text{Error}$

Reshaping arrays Two common numpy functions used in deep learning are `np.shape` and `np.reshape()`.

- `X.shape` is used to get the shape (dimension) of a matrix/vector **X** - `X.reshape(...)` is used to reshape **X** into some other dimension.

In computer science, an image is represented by a 3D array of shape (height, width, channel). However, when you read an image as the input of an algorithm you convert it to a vector of shape (height * width * channel, 1).

Exercise 2 Implement `image2vector()` function that takes an input of shape (height, width, 3) and returns a vector of shape (height * width * 3, 1).

Do not hardcode the dimensions of image as a constant. The dimensions can varies from different images.

```
[8]: # Grade cell - do not remove

def image2vector(image):
    """
    Convert image with 3 dimensions to become vector of (size, 1)
    """
    # height, width, channel = image.shape
    h, w, c = image.shape
    # print(image.shape)
    v = image.reshape(h * w * c, 1)
    # print(v.shape)
    # YOUR CODE HERE
    # raise NotImplementedError()

    return v
```

```
[9]: # test function - do not remove
image = np.array([[[ 0.67826139,  0.29380381],
                   [ 0.90714982,  0.52835647],
                   [ 0.4215251 ,  0.45017551]],

                  [[ 0.92814219,  0.96677647],
                   [ 0.85304703,  0.52351845],
                   [ 0.19981397,  0.27417313]],

                  [[ 0.60659855,  0.00533165],
                   [ 0.10820313,  0.49978937],
                   [ 0.34144279,  0.94630077]]])
```

```

print ("image2vector(image) = " + str(image2vector(image)))

import cv2

img = cv2.imread("img/lena.png")

print(img.shape)
vecimg = image2vector(img)
print(vecimg.shape)

assert image2vector(image).shape == (image.shape[0] * image.shape[1] * image.
↳shape[2], 1), "Dimension is incorrect"

```

```

image2vector(image) = [[0.67826139]
 [0.29380381]
 [0.90714982]
 [0.52835647]
 [0.4215251 ]
 [0.45017551]
 [0.92814219]
 [0.96677647]
 [0.85304703]
 [0.52351845]
 [0.19981397]
 [0.27417313]
 [0.60659855]
 [0.00533165]
 [0.10820313]
 [0.49978937]
 [0.34144279]
 [0.94630077]]
(512, 512, 3)
(786432, 1)

```

Expected Output: image2vector(image) = [[0.67826139]

```

[0.29380381]
[0.90714982]
[0.52835647]
[0.4215251 ]
[0.45017551]
[0.92814219]
[0.96677647]
[0.85304703]
[0.52351845]
[0.19981397]
[0.27417313]
[0.60659855]
[0.00533165]

```

```
[0.10820313]
[0.49978937]
[0.34144279]
[0.94630077]]
(512, 512, 3)
(786432, 1)
```

Matrix Normalization Normalization is a technique for machine learning and deep learning. It generally ensures a better performance since the gradient descent converges faster after the data is normalized. The normalization also controls the parameters not to overflow while updating the gradients. For example, we have a matrix $\mathbf{A}$

$$\mathbf{A} = \begin{bmatrix} 8 & 1 & 2 \\ 3 & 9 & 5 \end{bmatrix}$$

We can get the norm the matrix using `np.linalg.norm()` function. In our case, we find the norm along the rows. By using numpy library function, we can write the code as follows:

$$\|A\| = np.linalg.norm(\mathbf{A}, axis = 1)$$

and normalize the matrix by simply dividing the original matrix by the norm.

$$norm(\mathbf{A}) = \frac{\mathbf{A}}{\|\mathbf{A}\|}$$

Broadcasting Notice that numpy has a feature called broadcasting which allows the operation of matrices with different sizes by duplicating to get the same size. (In our case, it duplicates the norm matrix into two rows before doing the division)

$$\begin{bmatrix} 8 & 1 & 2 \\ 3 & 9 & 5 \end{bmatrix} / \begin{bmatrix} norm_0 \\ norm_1 \end{bmatrix} = \begin{bmatrix} 8 & 1 & 2 \\ 3 & 9 & 5 \end{bmatrix} / \begin{bmatrix} norm_0 & norm_0 & norm_0 \\ norm_1 & norm_1 & norm_1 \end{bmatrix}$$

Implement `normalizeRows()` to normalize the rows of a matrix. After applying this function to an input matrix $\mathbf{x}$, each row of $\mathbf{x}$ should be a vector of unit length (meaning length 1).

```
[10]: print(img.shape)
```

```
(512, 512, 3)
```

Example

```
[11]: # Grade cell - do not remove
def normalizeRows(x):
    """
    Implement a function that normalizes each row of the matrix x (to have unit
    length).
    """
    print("shape: ", np.linalg.norm(x, axis = 1).shape)
```

```

print("norm value: ", np.linalg.norm(x, axis = 1))
norm_x = x/np.linalg.norm(x, axis =1).reshape(-1, 1)
# YOUR CODE HERE
#     raise NotImplementedError()

return norm_x

```

[12]: # test function - do not remove

```

A = np.array([
    [8, 1, 2],
    [3, 9, 5]])

norm_A = normalizeRows(A)
print("normalizeRows(A) = " + str(norm_A))

sqr_A = norm_A * norm_A
sum_A = np.sum(sqr_A, axis = 1)
print("prove of A in each row", sum_A)

assert sum_A.shape == (2,), "normalize must do in row"
assert np.round(sum_A[0], 1) == 1 and np.round(sum_A[1], 1) == 1, "Normalize_
↪incorrect"

```

```

shape: (2,)
norm value: [ 8.30662386 10.72380529]
normalizeRows(A) = [[0.96308682 0.12038585 0.24077171]
 [0.27975144 0.83925433 0.4662524 ]]
prove of A in each row [1. 1.]

```

Expected Output: normalizeRows(A) = [[0.96308682 0.12038585 0.24077171]
[0.27975144 0.83925433 0.4662524]]
prove of A in each row [1. 1.]

4 Implement the L1 and L2 loss functions

The loss L1 and L2 are used to evaluate the performance of your model. The bigger your loss is, the more different your predictions $\hat{h}$ are from the true values y . In deep learning, Gradient Descent or Ascent is used to optimize models by minimizing the cost.

To assume loss function in L_1 , the L1 loss is defined as

$$L_1(\hat{y}, y) = \sum_{i=0}^m |y^{(i)} - \hat{y}^{(i)}|$$

Example Implement the numpy vectorized version of the L1 loss. use function `np.abs()` to apply the equation.

```
[12]: def L1(yhat, y):
#     loss = np.sum(np.abs(y - yhat))
    loss = np.abs(y-yhat).sum()
    # YOUR CODE HERE
#     raise NotImplementedError()
    return loss
```

```
[13]: # test function - do not remove

yhat = np.array([.9, 0.2, 0.1, .4, .9])
yhat2 = np.array([.1, 0.7, 0.4, 0.7, .8, 0.2])
y = np.array([1, 0, 0, 1, 1])
y2 = np.array([1, 0, 0, 1, 1, 0])
l1 = L1(yhat,y)
l2 = L1(yhat2,y2)
print("L1 of output 1 = " + str(l1))
print("L1 of output 2 = " + str(l2))

assert np.round(l1,1) == 1.1, "L1 loss is incorrect"
assert np.round(l2,1) == 2.7, "L1 loss is incorrect"
```

L1 of output 1 = 1.1

L1 of output 2 = 2.7

Expected Output:

L1 of output 1 = 1.1

L1 of output 2 = 2.7

To assume loss function in L_2 , the L2 loss is defined as

$$L_2(\hat{y}, y) = \sum_{i=0}^m (y^{(i)} - \hat{y}^{(i)})^2$$

Example Implement the numpy vectorized version of the L2 loss.

```
[14]: # Grade cell - do not remove

def L2(yhat, y):
    loss = np.square(y-yhat).sum()
    # YOUR CODE HERE
#     raise NotImplementedError()

    return loss
```

```
[15]: # test function - do not remove

yhat = np.array([.9, 0.2, 0.1, .4, .9])
yhat2 = np.array([.1, 0.7, 0.4, 0.7, .8, 0.2])
```



```

y = np.array([1, 0, 0, 1, 1])
y2 = np.array([1, 0, 0, 1, 1, 0])
l1 = L2(yhat,y)
l2 = L2(yhat2,y2)
print("L2 of output 1 = " + str(l1))
print("L2 of output 2 = " + str(l2))

assert np.round(l1,2) == 0.43, "L2 loss is incorrect"
assert np.round(l2,2) == 1.63, "L2 loss is incorrect"

```

L2 of output 1 = 0.43
L2 of output 2 = 1.6300000000000001

Expected Output:

L2 of output 1 = 0.43
L2 of output 2 = 1.6300000000000001

5 Break Down of a Neural Network Classification Training

A neural network training can be broken down into the following subtasks:

1. Data Preprocessing
2. Forward Propagation
3. Loss Function
4. Back Propagation
5. Parameters Update
6. Hyperparameter Tuning

We will implement these subtasks step by step from scratch in order to have a better understanding of the background process.

5.1 1. Data Preprocessing

This step includes:

- a. loading the input images and visualizing them
- b. Getting one hot encoded labels
- c. Normalization
- d. Splitting data to training, validation and test sets

Loading Images The dataset we are going to use in this training is the images of handwritten digits which we can get from `sklearn` library. Firstly, we have to import the required libraries as below:

```

[16]: import numpy as np
      from sklearn.datasets import load_digits
      import matplotlib.pyplot as plt

```

Then, load the dataset. and see the data and shape of X(inputs) and y(labels).

```
[17]: # Load data
data = load_digits()

y_indices = data.target # load the targets(labels)
X = np.matrix(data.data) # load the input images

print('y shape:', y_indices.shape)
print('X shape:', X.shape)

print('y data:', y_indices[0:15])
print('X data:', X[0:5])

data_size = X.shape[0]
x_area = X.shape[1]

y shape: (1797,)
X shape: (1797, 64)
y data: [0 1 2 3 4 5 6 7 8 9 0 1 2 3 4]
X data: [[ 0.  0.  5. 13.  9.  1.  0.  0.  0.  0. 13. 15. 10. 15.  5.  0.  0.
 3.
 15.  2.  0. 11.  8.  0.  0.  4. 12.  0.  0.  8.  8.  0.  0.  5.  8.  0.
 0.  9.  8.  0.  0.  4. 11.  0.  1. 12.  7.  0.  0.  2. 14.  5. 10. 12.
 0.  0.  0.  0.  6. 13. 10.  0.  0.  0.]
[ 0.  0.  0. 12. 13.  5.  0.  0.  0.  0.  0. 11. 16.  9.  0.  0.  0.  0.
 3. 15. 16.  6.  0.  0.  0.  7. 15. 16. 16.  2.  0.  0.  0.  0.  1. 16.
16.  3.  0.  0.  0.  0.  1. 16. 16.  6.  0.  0.  0.  0.  1. 16. 16.  6.
 0.  0.  0.  0.  0. 11. 16. 10.  0.  0.]
[ 0.  0.  0.  4. 15. 12.  0.  0.  0.  0.  3. 16. 15. 14.  0.  0.  0.  0.
 8. 13.  8. 16.  0.  0.  0.  0.  1.  6. 15. 11.  0.  0.  0.  1.  8. 13.
15.  1.  0.  0.  0.  9. 16. 16.  5.  0.  0.  0.  0.  3. 13. 16. 16. 11.
 5.  0.  0.  0.  0.  3. 11. 16.  9.  0.]
[ 0.  0.  7. 15. 13.  1.  0.  0.  0.  8. 13.  6. 15.  4.  0.  0.  0.  2.
 1. 13. 13.  0.  0.  0.  0.  0.  2. 15. 11.  1.  0.  0.  0.  0.  0.  1.
12. 12.  1.  0.  0.  0.  0.  0.  1. 10.  8.  0.  0.  0.  8.  4.  5. 14.
 9.  0.  0.  0.  7. 13. 13.  9.  0.  0.]
[ 0.  0.  0.  1. 11.  0.  0.  0.  0.  0.  0.  7.  8.  0.  0.  0.  0.  0.
 1. 13.  6.  2.  2.  0.  0.  0.  7. 15.  0.  9.  8.  0.  0.  5. 16. 10.
 0. 16.  6.  0.  0.  4. 15. 16. 13. 16.  1.  0.  0.  0.  0.  3. 15. 10.
 0.  0.  0.  0.  0.  2. 16.  4.  0.  0.]]
```

The shape of X is (1797, 64) which means we have 1797 samples of data in (1, 64) long vector. Since each sample represents 1 D vector and the actual size of an image is 8x8, we need to convert the size of input X into 8x8.

Visualize Image

Exercise 3 Write a function to convert the X data of one image to be image size 8x8. Check the size of input vector to convert image and verify after conversion.

Hint: use `np.reshape()`

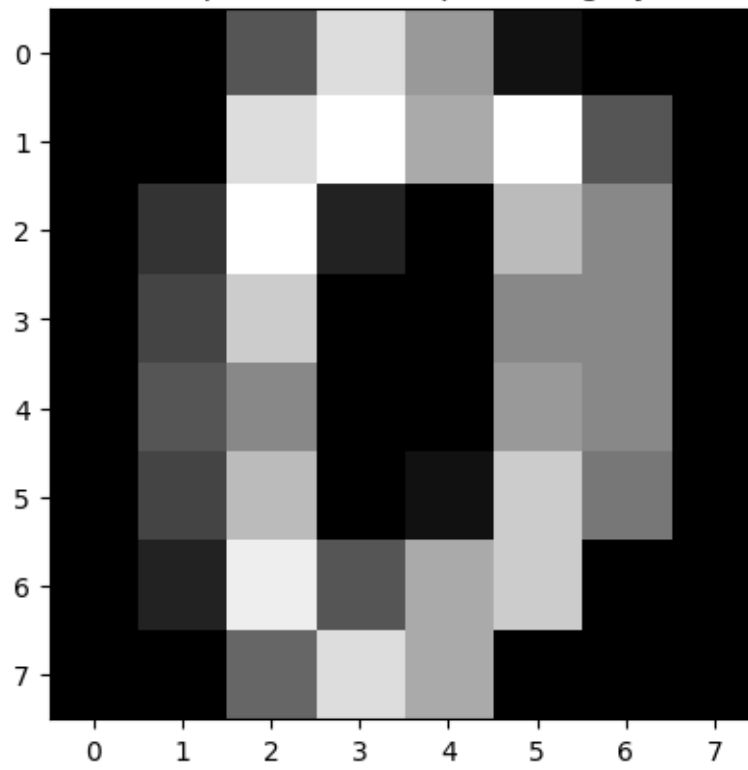
```
[18]: # Grade cell - do not remove
```

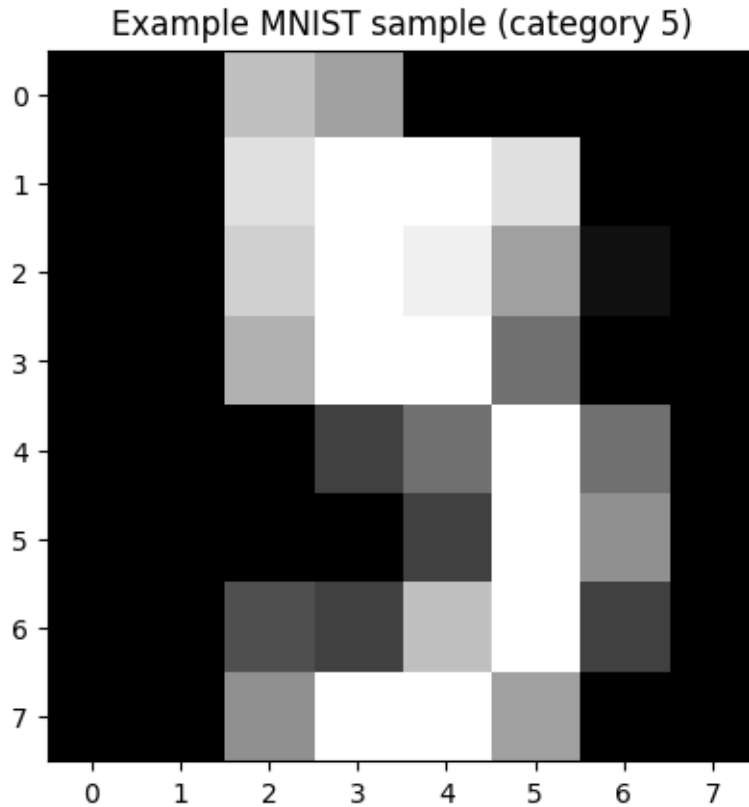
```
def convert_image(X_one_image):  
    img = np.array(X_one_image)  
    # YOUR CODE HERE  
    changesize=int(math.sqrt(img.size))  
    img=np.reshape(img,(changesize,changesize))  
    return img
```

```
[19]: # test function - do not remove
```

```
img_0 = convert_image(X[0])  
plt.imshow(img_0, 'gray')  
plt.title('Example MNIST sample (category %d)' % y_indices[0])  
plt.show()  
  
img_5 = convert_image(X[5,:])  
plt.imshow(img_5, 'gray')  
plt.title('Example MNIST sample (category %d)' % y_indices[5])  
plt.show()  
  
test_v = np.empty([1,256])  
test = convert_image(test_v)  
  
assert img_0.shape == (8,8) and img_5.shape == (8,8), 'Image reshape is_  
↳incorrect'  
assert test.shape == (16,16), 'Image reshape is incorrect'  
assert img_0[3,6] == X[0, 30] and img_5[4,2] == X[5, 34], 'Image reshape is_  
↳incorrect'
```

Example MNIST sample (category 0)





Expected Output:

One Hot Encoding As you can see, the output y is index value. To use the value for classify in deep learning, you need to convert it to one hot matrix.

One hot encoding is a process by which categorical variables are converted into a form that could be provided to ML algorithms to do a better job in prediction.

To do so, we need to convert the index value to the following format:

$$0 \rightarrow [1, 0, 0, 0, 0, 0, 0, 0, 0]$$

$$1 \rightarrow [0, 1, 0, 0, 0, 0, 0, 0, 0]$$

$$2 \rightarrow [0, 0, 1, 0, 0, 0, 0, 0, 0]$$

$$3 \rightarrow [0, 0, 0, 1, 0, 0, 0, 0, 0]$$

$$4 \rightarrow [0, 0, 0, 0, 1, 0, 0, 0, 0]$$

$$5 \rightarrow [0, 0, 0, 0, 0, 1, 0, 0, 0]$$

$$6 \rightarrow [0, 0, 0, 0, 0, 0, 1, 0, 0]$$

$$7 \rightarrow [0, 0, 0, 0, 0, 0, 0, 1, 0]$$

8 $\rightarrow$ [0, 0, 0, 0, 0, 0, 0, 0, 1, 0]
 9 $\rightarrow$ [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]

Exercise 4 Implement one-hot vector function

[20]: *# Grade cell - do not remove*

```
# def convert_to_one_hot(y, onehot_size):
#     y_vect = np.array(y)
#     # YOUR CODE HERE
#     raise NotImplementedError()
#     return y_vect

def convert_to_one_hot(y, onehot_size):
    y_vect = np.array(y)
    one_hot = np.zeros((y_vect.size, onehot_size), dtype=float)
    one_hot[np.arange(y_vect.size), y_vect] = 1
    return one_hot
```

[21]: *# test function - do not remove*

```
y = convert_to_one_hot(y_indices, 10)
print(y.shape)
print(y[3])
assert y.shape[1] == 10 and y.shape[0] == 1797, "One hot size is incorrect"
assert y[14, 8] == 0 and y[177, 1] == 1, "One hot value is incorrect"
```

```
(1797, 10)
[0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]
```

Expected Output:

```
(1797, 10)
[0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]
```

[22]: `print(X.shape)`

```
(1797, 64)
```

Normalize Input Feature Now, change the input $\mathbf{x}$ to be a normalized input vector. The normalization equation is:

$$\text{norm}(\mathbf{x}) = \frac{\mathbf{x} - \tilde{\mathbf{x}}}{\sigma}$$

where,

$\tilde{\mathbf{x}}$ = Mean

σ = Standard Deviation

This is called z-score normalization.

Exercise 5 Write a normalization code. If some values are nan, please change them to be zero, using np.nan_to_num()

```
[23]: # Grade cell - do not remove
```

```
def normalize(X):
    mean = np.mean(X)
    std = np.std(X)
    # print(mean)
    # print(std)

    # YOUR CODE HERE
    X_vect=np.nan_to_num(X)
    if std==0:
        XX = (X_vect-mean)
    else:
        XX = (X_vect-mean)/std

    return XX
```

```
[24]: # test function - do not remove
```

```
XX = normalize(X)

print(XX[0])

assert XX.shape == X.shape, "Normalize function is incorrect"
assert np.max(XX[0]) < 2 and np.min(XX[0]) > -2, "Data is not normalize"
```

```
[[-0.8117562 -0.8117562  0.01925204  1.34886521  0.68405863 -0.64555455
 -0.8117562 -0.8117562 -0.8117562 -0.8117562  1.34886521  1.68126851
  0.85026027  1.68126851  0.01925204 -0.8117562 -0.8117562 -0.31315126
  1.68126851 -0.4793529 -0.8117562  1.01646192  0.51785698 -0.8117562
 -0.8117562 -0.14694961  1.18266357 -0.8117562 -0.8117562  0.51785698
  0.51785698 -0.8117562 -0.8117562  0.01925204  0.51785698 -0.8117562
 -0.8117562  0.68405863  0.51785698 -0.8117562 -0.8117562 -0.14694961
  1.01646192 -0.8117562 -0.64555455  1.18266357  0.35165533 -0.8117562
 -0.8117562 -0.4793529  1.51506686  0.01925204  0.85026027  1.18266357
 -0.8117562 -0.8117562 -0.8117562 -0.8117562  0.18545368  1.34886521
  0.85026027 -0.8117562 -0.8117562 -0.8117562 ]]
```

Expected Output:

```
[[ 0. -0.33501649 -0.04308102 0.27407152 -0.66447751 -0.84412939
 -0.40972392 -0.12502292 -0.05907756 -0.62400926 0.4829745 0.75962245
 -0.05842586 1.12772113 0.87958306 -0.13043338 -0.04462507 0.11144272
 0.89588044 -0.86066632 -1.14964846 0.51547187 1.90596347 -0.11422184
 -0.03337973 0.48648928 0.46988512 -1.49990136 -1.61406277 0.07639777
 1.54181413 -0.04723238 0. 0.76465553 0.05263019 -1.44763006
 -1.73666443 0.04361588 1.43955804 0. -0.06134367 0.8105536]
```

```
0.63011714 -1.12245711 -1.06623158 0.66096475 0.81845076 -0.08874162
-0.03543326 0.74211893 1.15065212 -0.86867056 0.11012973 0.53761116
-0.75743581 -0.20978513 -0.02359646 -0.29908135 0.08671869 0.20829258
-0.36677122 -1.14664746 -0.5056698 -0.19600752]]
```

Splitting Data In a machine learning training, it is necessary to split the existing data into three different types:

1. Training set - Data that is used for training the model
2. Validation set - Data that is used for validating the model performance throughout the training at each epoch or iteration. The validation data is used to verify the model is learning and not to overfit or underfit the training data.
3. Testing set - Data that is used in a process called inferencing. The test set is totally unseen by the model during training and validation process. Higher performance of the model on the testing data means the model is robust and generalizable to unseen data.

The process of splitting the training, validate, and test set needs the following criteria:

1. The data need to be random before split.
2. The validate and test set can be from the same distribution as training data, however the same data must be strictly in only one split set(train or validation or test).
3. More data is almost always helps, however the consistency and reliability of data are key importances when selection of data(garbage in garbage out).

Normally, we should split data in percentage. However, this is not fixed. You can adjust.

- 60% training, 20% validate, and 20% test
- 80% training, 10% validate, and 10% test
- For the very low data (~1000 data), we could use validate and test set in the same data.

Exercise 6 Random split the train set to be 60% of data and the rest is the test set.

Hint: use `np.arange()` for set index number from 0 to `data_size`. Random index can do by using `random.shuffle()`. Please make sure that you are using normalized data to split.

```
[25]: percent_train = 0.6
data_size = len(data.data)    # assuming data is already normalized

# arange index number from 0 to data_size
idx = np.arange(data_size)

# random shuffle idx (1 line)
np.random.seed(7)
np.random.shuffle(idx)

# calculate number of training set
m_train = int(percent_train * data_size)

# split train_idx and test_idx
train_idx = idx[0:m_train]
```



```

test_idx = idx[m_train:]

# split to X_train and X_test
X_train = data.data[train_idx]
X_test = data.data[test_idx]

# split to y_train y_test and y_test_indices
# (assuming you have labels y with same length as data)
y_train = y[train_idx]
y_test = y[test_idx]
y_test_indices = test_idx

print("Train X:", X_train)
print("Test X:", X_test)
print("Train y:", y_train)
print("Test y:", y_test)
print("Test indices:", y_test_indices)

```

```

Train X: [[ 0.  0.  0. ... 15.  5.  0.]
 [ 0.  0.  2. ...  5.  0.  0.]
 [ 0.  0. 11. ...  0.  0.  0.]

```

...

```

 [ 0.  0.  5. ...  0.  0.  0.]
 [ 0.  4. 13. ...  7.  0.  0.]
 [ 0.  0.  6. ...  0.  0.  0.]]

```

```

Test X: [[ 0.  0.  2. ...  8.  0.  0.]
 [ 0.  0.  0. ...  9.  0.  0.]
 [ 0.  0.  8. ...  6.  0.  0.]

```

...

```

 [ 0.  0.  8. ... 16. 12.  5.]
 [ 0.  2. 13. ... 12.  1.  0.]
 [ 0.  1. 10. ...  4.  0.  0.]]

```

```

Train y: [[0. 0. 0. ... 0. 0. 0.]
 [1. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]

```

...

```

 [0. 0. 0. ... 1. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 1. 0. 0.]]

```

```

Test y: [[0. 0. 0. ... 0. 0. 0.]
 [0. 1. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]

```

...

```

 [0. 1. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]]

```

```

Test indices: [1683  466 1320 1173  277 1069 1017 1188  722 1741  786 1379 1429
1229

```

1170	906	563	388	22	1098	124	1599	773	880	50	1252	1304	1105
703	482	1703	66	1537	733	1595	801	1380	1371	907	31	417	881
632	1394	1578	64	1677	77	491	1266	1360	532	65	1513	296	591
767	724	626	1318	1735	1441	251	1626	1493	1696	982	1584	706	487
273	1345	1494	963	877	1647	1621	411	667	149	644	1435	218	239
1131	792	191	333	1190	962	1187	1739	1478	1446	582	1701	1286	839
627	323	1469	1160	778	1076	939	437	119	1366	1395	1026	1303	76
1053	900	371	1568	1762	941	577	819	1313	1332	1547	1238	886	968
785	1374	1438	301	689	812	1061	236	1115	1226	499	1483	1079	794
1593	990	1263	934	387	390	743	352	1217	235	1526	480	567	1140
1750	1664	1743	120	1543	1024	1086	926	515	223	1321	613	843	52
1651	485	1707	818	155	600	1433	248	761	1633	1130	542	793	1141
1302	628	150	1768	291	381	686	1224	897	854	1071	1781	1582	1654
891	1785	615	154	1193	1531	1779	322	1381	1282	82	512	1361	929
1459	10	831	32	1353	413	580	1383	1698	614	1045	1014	89	1587
601	1237	133	555	1031	1399	320	1583	1211	1552	634	552	1055	1392
796	1489	1264	1577	174	766	245	1080	1551	145	90	682	310	93
1401	924	1511	1035	243	1111	214	1197	998	853	425	1575	1142	1143
226	1776	0	401	1103	84	162	114	670	1174	1363	889	625	1748
952	807	258	624	826	234	135	616	1749	836	842	959	592	1314
1291	343	1477	1408	293	364	178	659	996	14	1110	165	1678	1359
59	709	676	1378	780	424	292	1065	1704	329	429	1012	79	1052
937	834	749	890	254	1549	1747	1407	1337	1108	136	1135	1218	852
1202	723	859	969	496	950	805	129	337	1780	608	1148	195	1250
1292	1764	909	584	103	1653	1668	719	884	776	849	894	227	1680
1325	1299	126	405	775	821	711	1687	1060	1163	303	267	545	212
896	1222	916	675	1289	814	1265	342	549	1410	1214	966	1230	295
695	1452	73	605	104	645	1404	1752	1040	439	1019	91	1112	1570
908	1474	102	238	1525	942	1270	938	1518	1181	5	1625	965	18
1430	121	661	1510	274	433	811	1300	1335	1669	1688	181	184	406
865	520	1003	380	1434	1393	1424	427	78	1009	1760	607	1597	1777
1631	1057	1440	1514	1091	1101	1129	1235	1580	958	559	846	690	1519
1729	1667	436	490	1782	1643	1387	69	808	1184	11	144	917	1541
1451	639	176	994	440	521	1767	1733	1004	1255	1234	143	753	519
1788	1322	822	428	612	1505	1032	156	539	1530	1460	1290	461	576
1362	255	979	1185	1006	1521	560	208	910	1311	1	1365	269	265
914	230	1763	1119	1758	864	1066	873	1738	398	1634	45	161	991
727	629	288	1613	1059	1744	575	850	1339	911	1254	1627	932	1100
1207	247	476	85	469	225	774	331	233	588	475	1284	599	421
182	988	1000	1356	1462	49	1675	1029	529	1258	486	1742	497	1646
106	443	1512	646	1036	726	190	282	29	169	1001	1204	100	197
495	647	987	44	710	373	678	566	1672	847	460	1437	1155	39
434	973	1239	1007	1579	1590	1498	456	260	35	827	309	276	636
654	622	844	393	107	349	524	784	47	203	1792	1655	21	402
1107	1154	1439	989	741	1310	1147	33	787	1540	923	1043	1723	1243
579	481	931	356	1685	1248	759	92	1175	1415	1341	1472	803	745
1546	279	1607	764	604	1774	1373	1398	400	1728	257	944	1096	1219
1507	1161	494	772	550	1486	201	312	290	1213	763	1136	949	183

```
1176 1349 1727 1068 1468 1030 250 823 1099 1536 940 391 1456 1092
1191 904 730 554 366 345 1608 535 1422 1209 1372 1639 919 211
1603 502 537 1220 175]
```

```
[26]: # test function - do not remove

assert X_train.shape[0] == int(percent_train * data_size), "training size is_
↳incorrect"
assert data_size - m_train == X_test.shape[0], "test size is incorrect"
assert train_idx[0] != 0 and train_idx[25] != 25, "training indices are not_
↳shuffled"
assert X_train.shape == (m_train, XX.shape[1]) and y_train.shape[0] == m_train
assert X_test.shape == (data_size - m_train, XX.shape[1]) and y_test.shape[0]_
↳== data_size - m_train and y_test_indices.shape[0] == data_size - m_train
```

```
[ ]:
```